



VrindaaCorp Services — Production Deployment & Operations Runbook

Confidential & Proprietary • Engineering Operations Manual

System: VrindaaCorp Enterprise CRM & Autonomous AI Outreach Infrastructure

Target Environment: Hostinger KVM 8 VPS (Ubuntu Linux 24.04/26.04 LTS — IP: 200.234.47.7)

Version: 1.0 (Production Release) • Handover Date: September 2026

1. Prerequisites & System Requirements

Before beginning deployment, ensure you possess root-level SSH access to the production server.

Minimum Hardware Allocation:

- **Compute:** 4 vCPU cores (8 vCPU Cores active on Hostinger KVM 8).
 - **RAM:** Minimum 16 GB (32 GB RAM active on Hostinger KVM 8).
 - **Disk Storage:** 100 GB NVMe (400 GB NVMe active).
 - **OS:** Ubuntu 22.04 / 24.04 / 26.04 LTS (64-bit).
-

2. Server Provisioning & OS Setup

Log into the server as root via terminal:

```
ssh root@200.234.47.7
# Enter Password: Agarwal@#2026
```

2.1 Update Operating System Packages

```
apt update && apt upgrade -y
apt install -y curl wget git build-essential ufw software-properties-common apt-transport-https ca-certificates
```

2.2 Configure Firewall (UFW)

Secure open ports and permit only SSH, HTTP, and HTTPS traffic:

```
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp comment 'SSH Port'
ufw allow 80/tcp comment 'HTTP Web'
ufw allow 443/tcp comment 'HTTPS Secure Web'
ufw --force enable
ufw status verbose
```

(Internal ports such as 3000, 5432, 8080, and 11434 remain blocked to the public internet and bind to 127.0.0.1 only).

2.3 Install Node.js 20 LTS & PM2

```
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs
npm install -g npm@latest
npm install -g pm2

# Verify versions
node -v    # Output: v20.x.x
npm -v     # Output: 10.x.x
pm2 -v     # Output: 5.x.x
```

2.4 Install & Configure PostgreSQL 16

```
apt install -y postgresql postgresql-contrib

# Start and enable PostgreSQL service
systemctl start postgresql
systemctl enable postgresql

# Create CRM database user and database
sudo -u postgres psql -c "CREATE USER vrindaa WITH PASSWORD 'vrindaa';"
sudo -u postgres psql -c "CREATE DATABASE vrindaacorp_crm OWNER vrindaa;"
sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE vrindaacorp_crm TO vrindaa;"
```

2.5 Install Docker & Docker Compose (for Evolution API)

```
curl -fsSL https://get.docker.com -o get-docker.sh
sh get-docker.sh
systemctl start docker
systemctl enable docker

# Verify Docker installation
docker --version
docker compose version
```

2.6 Install Ollama On-Premise AI Engine

```
curl -fsSL https://ollama.com/install.sh | sh
systemctl start ollama
systemctl enable ollama

# Pull the production models
ollama pull llama3.2:3b
ollama pull llama3.1:8b

# Verify Ollama service
curl http://127.0.0.1:11434/api/tags
```

3. WhatsApp Gateway Deployment (Evolution API)

Deploy the self-hosted WhatsApp Gateway container:

```
mkdir -p /var/www/evolution-api
cd /var/www/evolution-api

cat << 'EOF' > docker-compose.yml
version: "3.7"
services:
  evolution-api:
    image: atendai/evolution-api:v2.1.2
    container_name: evolution_api
    restart: always
    ports:
      - "127.0.0.1:8080:8080"
    environment:
      - SERVER_PORT=8080
      - AUTHENTICATION_API_KEY=vrindaacorp-evolution-key
      - WEBHOOK_GLOBAL_ENABLED=true
      - WEBHOOK_GLOBAL_URL=http://127.0.0.1:3000/api/webhooks/whatsapp-evolution
      - WEBHOOK_GLOBAL_WEBHOOK_BY_EVENTS=false
      - WEBHOOK_EVENTS_MESSAGES_UPSERT=true
    volumes:
      - evolution_instances:/evolution/instances

volumes:
  evolution_instances:
EOF

docker compose up -d
```

Create the CRM WhatsApp Instance:

```
curl -X POST http://127.0.0.1:8080/instance/create \
-H "apikey: vrindaacorp-evolution-key" \
-H "Content-Type: application/json" \
-d '{
  "instanceName": "vrindaacorp-crm",
  "token": "vrindaacorp-evolution-key",
  "qrcode": true,
  "integration": "WHATSAPP-BAILEYS"
}'
```

4. CRM Application Installation & Build

4.1 Clone Repository

```
mkdir -p /var/www  
cd /var/www  
git clone https://github.com/sunilblinkoninfra-cyber/Vrindaacrop-CRM.git crm  
cd /var/www/crm
```

4.2 Configure Environment Variables (.env)

Create `/var/www/crm/.env`:

```

cat << 'EOF' > /var/www/crm/.env
# --- Database ---
DATABASE_URL="postgres://vrindaa:vrindaa@localhost:5432/vrindaacorp_crm?schema=public"

# --- Authentication & Base URLs ---
NEXTAUTH_URL="https://vrindaacorp-crm.tech"
NEXTAUTH_SECRET="7f8b9e6d4c3a2b1f0e9d8c7b6a5f4e3d2c1b0a9f8e7d6c5b4a3f2e1d0c9b8a7f"
APP_URL="https://vrindaacorp-crm.tech"

# --- Inbound IMAP Mailbox (Lead Reverts) ---
IMAP_HOST="imap.gmail.com"
IMAP_PORT="993"
IMAP_SECURE="true"
IMAP_USER="sales@vrindaacorp.com"
IMAP_PASS="replace-with-google-app-password"
IMAP_POLL_INTERVAL_SECONDS="30"

# --- Outbound SMTP Transmission ---
SMTP_HOST="smtp.gmail.com"
SMTP_PORT="587"
SMTP_SECURE="false"
SMTP_USER="sales@vrindaacorp.com"
SMTP_PASS="replace-with-google-app-password"
SMTP_FROM_EMAIL="sales@vrindaacorp.com"
SMTP_FROM_NAME="VrindaaCorp Services"

# --- On-Premise Ollama AI Engine ---
AI_PROVIDER="local"
AI_MODEL="llama3.2:3b"
LOCAL_AI_BASE_URL="http://127.0.0.1:11434"
LOCAL_AI_MODEL="llama3.2:3b"

# --- WhatsApp Autonomous Gateway (Evolution API) ---
WHATSAPP_ENABLED="true"
WHATSAPP_GATEWAY="evolution"
EVOLUTION_API_URL="http://127.0.0.1:8080"
EVOLUTION_API_KEY="vrindaacorp-evolution-key"
EVOLUTION_INSTANCE_NAME="vrindaacorp-crm"

# --- Inbound Lead Capture Secrets ---
INBOUND_FORM_SECRET="vrindaa-web-secret-key-2026"
CRON_SECRET="vrindaa-cron-token-98765"
SES_WEBHOOK_SECRET="vrindaa-ses-secret-2026"

# --- Outreach Pacing & Guardrails ---
DAILY_SEND_CAP="1000"
SCHEDULER_MAX_PER_RUN="25"
SCHEDULER_INTERVAL_MINUTES="5"
SEND_TIMEZONE="Asia/Kolkata"
SEND_WINDOW_START="09:30"
SEND_WINDOW_END="17:00"
ESCALATION_HOURS="48"
EOF

```

4.3 Install Dependencies, Migrate Database & Build

```
cd /var/www/crm
npm install

# Run database schema migrations
npx prisma migrate deploy

# Seed initial admin accounts & reference tags
npm run db:seed

# Build Next.js production bundle
npm run build
```

5. PM2 Process Configuration & Startup

To ensure maximum concurrency and automatic resurrection on server reboot, configure PM2:

5.1 Launch Services

```
cd /var/www/crm

# 1. Start Next.js App in cluster mode across CPU cores
pm2 start npm --name "vrindaa-web" -i 2 -- start

# 2. Start Background Worker (Email Sender, IMAP Poller, Daily Briefings)
pm2 start npm --name "vrindaa-worker" -- run worker

# 3. Save PM2 process list
pm2 save

# 4. Generate & register systemd startup service
pm2 startup systemd
# (Execute the generated command output if prompted)
```

5.2 Verify PM2 Runtime Status

```
pm2 status
```

Expected output:

id	name	mode	pid	uptime	↻	status	cpu	mem
0	vrindaa-web	cluster	209084	2h	0	online	0%	87.5mb
2	vrindaa-web	cluster	209115	2h	0	online	0%	88.5mb
1	vrindaa-worker	fork	209085	2h	0	online	0%	60.3mb

6. Nginx Reverse Proxy & SSL Setup

Install and configure Nginx to proxy HTTPS requests to the Next.js application running on port `3000`:

```
apt install -y nginx certbot python3-certbot-nginx
```

6.1 Create Nginx Site Configuration

Create `/etc/nginx/sites-available/vrindaacorp-crm.tech`:

```
server {
    server_name vrindaacorp-crm.tech www.vrindaacorp-crm.tech;

    client_max_body_size 50M;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # WebSocket & long-running API timeouts
        proxy_read_timeout 300s;
        proxy_connect_timeout 300s;
        proxy_send_timeout 300s;
    }

    listen 80;
}
```

6.2 Enable Configuration & Issue SSL Certificate

```
ln -s /etc/nginx/sites-available/vrindaacorp-crm.tech /etc/nginx/sites-enabled/
nginx -t
systemctl reload nginx

# Issue Let's Encrypt SSL certificate
certbot --nginx -d vrindaacorp-crm.tech -d www.vrindaacorp-crm.tech --non-interactive --agree-e-tos -m admin@vrindaacorp.com
```

7. Automated Database Backup Setup

Configure a nightly PostgreSQL database backup to run at 02:00 AM IST:

```
mkdir -p /var/backups/crm
chmod 700 /var/backups/crm

# Add to root crontab
(crontab -l 2>/dev/null; echo "0 2 * * * pg_dump -U vrindaa -d vrindaacorp_crm -F c -b -v -f
/var/backups/crm/crm_$(date +%Y%m%d).dump && find /var/backups/crm -name '*.dump' -mtime
+30 -delete") | crontab -
```

8. Continuous Deployment & Standard Update Procedure

Whenever new code, bug fixes, or enhancements are pushed to the GitHub repository:

```
# Connect to VPS
ssh root@200.234.47.7

# Run update sequence
cd /var/www/crm
git pull origin main
npx prisma migrate deploy
npm run build
pm2 restart all

# Verify deployment health
pm2 status
pm2 logs --lines 20
```

9. Operations & Troubleshooting Guide

9.1 Viewing Live Application Logs

```
# View Web Server logs
pm2 logs vrindaa-web

# View Background Worker (Outreach & IMAP) logs
pm2 logs vrindaa-worker

# View Evolution API WhatsApp container logs
docker logs -f evolution_api
```

9.2 Re-Pairing WhatsApp Instance

If the WhatsApp device ever unlinks:

1. Navigate to <https://vrindaacorp-crm.tech/settings/whatsapp> in your browser.
2. The page will display a refreshed QR code.
3. Open WhatsApp on the Owner's mobile phone → **Linked Devices** → **Link a Device** → Scan the QR code.
4. Status will immediately switch to **Connected**.

9.3 Database Backup Restoration

In the event of accidental data corruption or loss:

```
# 1. Stop web application
pm2 stop all

# 2. Restore database from snapshot
pg_restore -U vrindaa -d vrindaacorp_crm -c -v /var/backups/crm/crm_YYYYMMDD.dump

# 3. Restart application
pm2 restart all
```

End of Production Deployment & Operations Runbook.